

Visualização e Iluminação

Apresentação Trabalho Prático

Ivan Ribeiro, Francisco Ferreira e Júlio Pinto

***Pathtracer* - Raynaldo**

- ***Rust***
- ***WGPU***
- ***Winit***
- ***Egui***
- ***Embree (Intel)***

Segurança, *performance*, controlo sobre a *pipeline* de renderização, *APIs* modernas

- *Multithreading*
- Carregamento de cenas de *OBJ*
- Janela interativa com *pathtracer* progressivo
- *Tonemapping*
- Estrutura de aceleração (*BVH*)
- ~~*Participating Media*~~

Três componentes fundamentais integrados:

- **Winit**: Sistema de janelas e eventos
- **WGPU**: *Renderer* na GPU (*Vulkan/Metal/DirectX 12*)
- **Egui**: *Interface* gráfica de modo imediato
- **Renderer**: *Output* do raytracer

```
struct RenderState {  
    canvas: AccumulationCanvas, // (r_sum, g_sum, b_sum, sample_count)  
    current_render_pixel: usize,  
    pixel_orders: [Vec<usize>; 5], // Ordem aleatória de renderização  
}
```

- Renderização em lotes de 10.000 píxeis
- Processamento paralelo via **Rayon**
- Orçamento de tempo por *frame* configurável
- 5 ordens de renderização diferentes para evitar padrões

```
while !state.render_state.is_finished(state.samples_per_pixel)
    && instant.elapsed() < Duration::from_millis(state.time_budget_ms)
{
    // Processamento paralelo de lote de pixels
    state.render_state.get_current_pixel_order()
        [state.render_state.current_render_pixel..end]
        .par_iter()
        .for_each(|&index_in_buffer| {
            let x = index_in_buffer as u32 % width;
            let y = index_in_buffer as u32 / width;
            let color = state.scene.render_sample(x, y, state.max_ray_depth);

            state.render_state.canvas.increment_pixel(x, y, color);
        });
}
```

@vertex

```
fn vs_main(@builtin(vertex_index) vertex_index: u32) -> VertexOutput {  
    // Gerar triângulo de acordo com o índice do vértice  
    let x: f32 = f32((vertex_index << 1u) & 2u) * 2.0 - 1.0;  
    let y: f32 = f32(vertex_index & 2u) * 2.0 - 1.0;  
    out.position = vec4<f32>(x, y, 0.0, 1.0);  
    out.uv = (vec2<f32>(x, y) + vec2<f32>(1.0)) * 0.5;  
    return out;  
}
```

@fragment

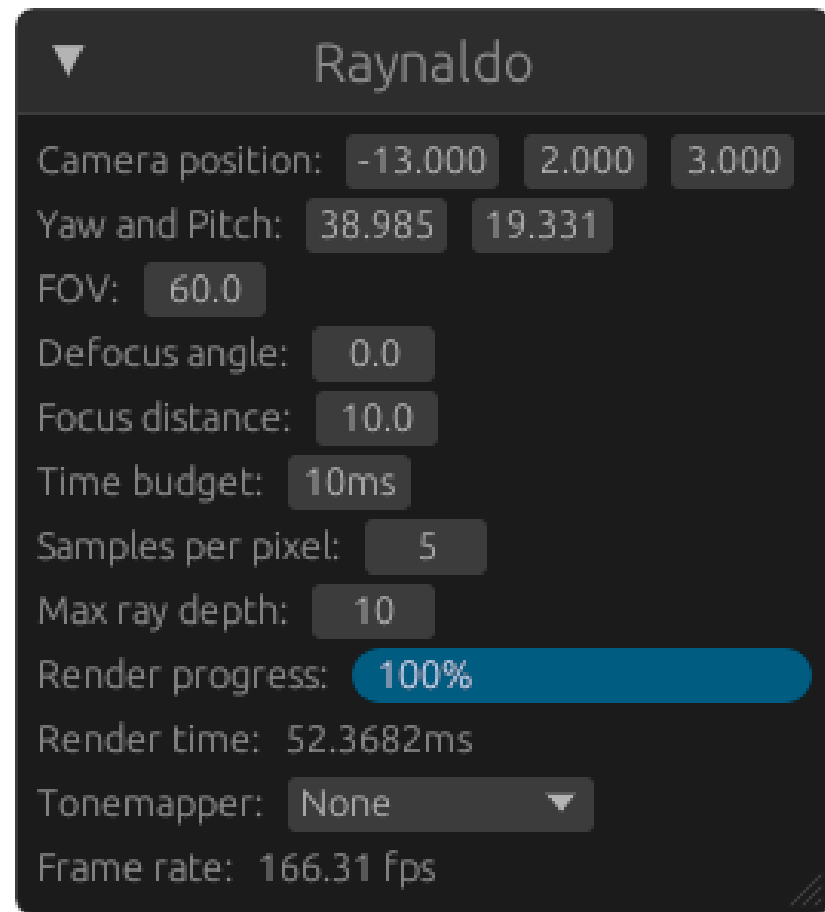
```
fn fs_main(input: VertexOutput) -> @location(0) vec4<f32> {  
    return textureSample(tex, tex_sampler, input.uv);  
}
```


Controlo da Câmara (FPV):

- *WASD*: movimento horizontal e frontal
- Espaço/*Shift*: subir/descer
- *Mouse*: rotação *yaw/pitch*

Interface Egui:

- Posição e orientação da câmara
- *FOV* e parâmetros de desfoque
- *Samples* por pixel e profundidade de raios
- *Tonemapping* e estatísticas de *performance*



```
[camera]
position = [1.85, 1.85, -4.0]
yaw = 90.0
pitch = 0.0
fov = 60.0
```

```
[environment]
type = "sky"
```

```
[[geometry]]
type = "sphere"
center = [0.0, 0.0, 0.0]
radius = 1.0
material = "lambertian"
texture = "solid"
color = [0.7, 0.3, 0.3, 1.0]
```

- **Esferas:** *center* + *radius*
- **Quads:** *origin* + vetores *u*, *v*
- **Malhas de Triângulos:** ficheiros *OBJ* ou definição implícita
- **Caixas Orientadas:** *origin* + vetores *u*, *v*, *w*

Deserialização automática via **Serde**

- **Lambertiano:** superfícies difusas (cor sólida, xadrez, texturas)
- **Metálico:** *albedo* + *fuzziness* (rugosidade)
- **Dielétrico:** materiais transparentes (índice de refração)
- **Emissivo:** fontes de luz (cor + intensidade)

Reinhard Simple: $V_i = \frac{C_i}{1+C_i}$

Reinhard com Saturação H&K: Modificação do *Reinhard*

Aplicado na *CPU* antes da transferência para *GPU*

Problema: Teste de interseção para cada triângulo é custoso

Solução: Estrutura de aceleração BVH via Embree

Embree já tem suporte nativo para esferas e *meshes* de triângulos

No entanto, não tem suporte para *quads* e *boxes*.

Solução: Fazer conversão de *quads* e *boxes* para *meshes* de triângulos.

Outro problema: *Embree* não devolve coordenadas *UVs* corretamente.

Solução: Calcular *UVs* manualmente a partir do ponto de interseção.

Ganhos de *Performance*:

- *Dragon* 8k: 36.5s → 0.096s
- *Dragon* 80k: 355.6s → 0.112s

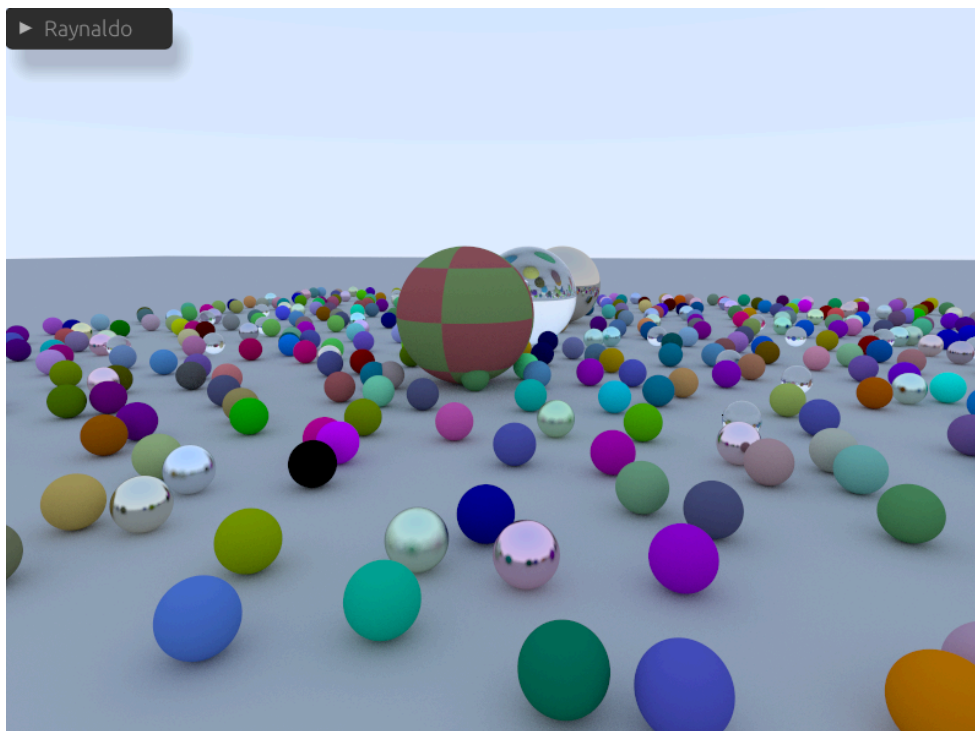


Figura 1: Cena com esferas

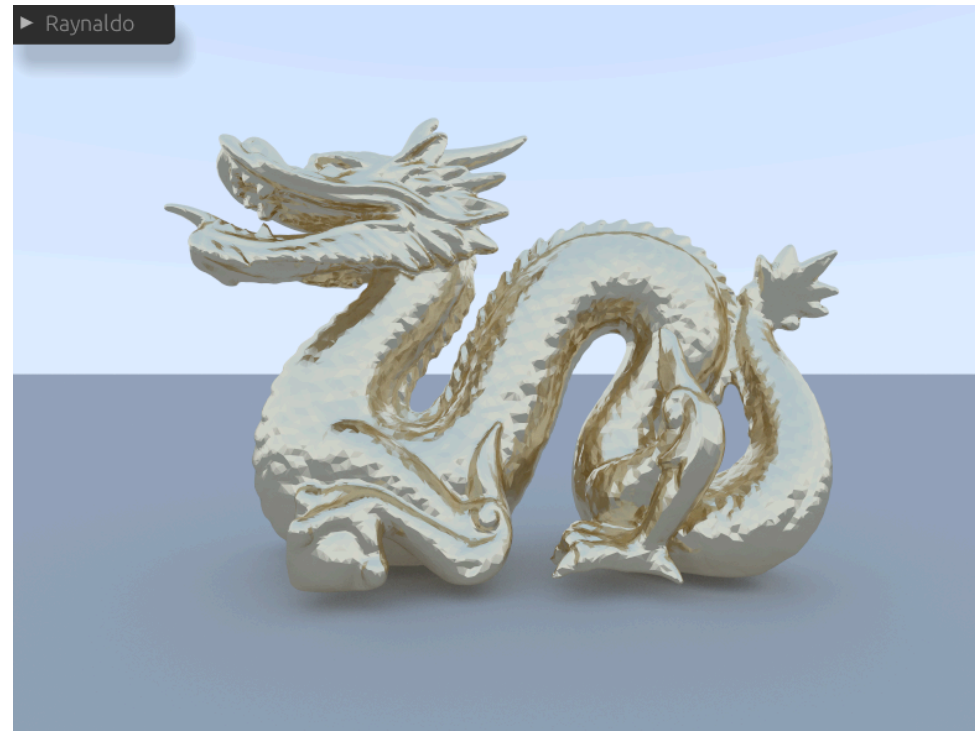


Figura 2: Dragão de *Stanford*

Todas as imagens renderizadas com 2000 *SPP*

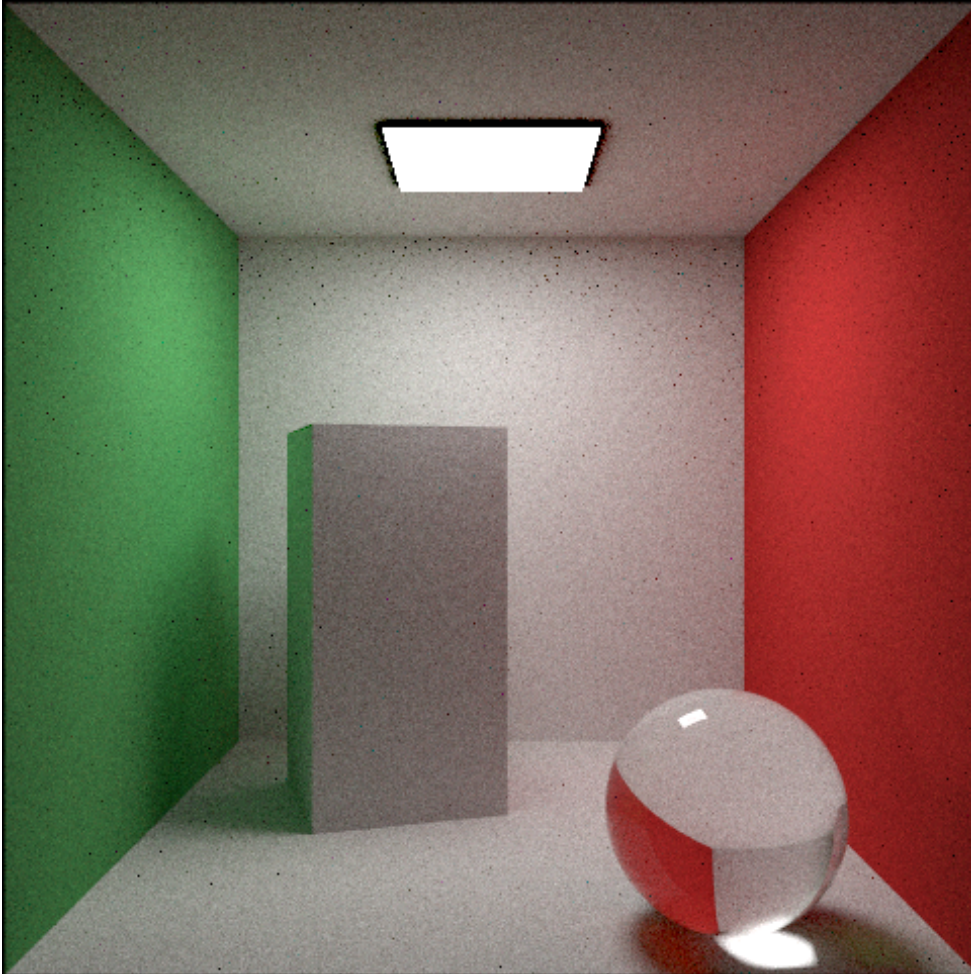


Figura 3: *Cornell Box* clássica



Figura 4: *Cornell Box* com texturas

Objetivos Alcançados:

- *Pathtracer* funcional e eficiente em *Rust*
- *Interface* interativa com renderização progressiva
- Sistema completo de materiais e geometrias
- Aceleração significativa com *BVH*

Trabalho Futuro:

- Suporte completo para materiais *OBJ*
- Redução de ruído
- Edição de mundo na *UI*
- *Export* de imagens

Demonstração
